Functions

1. What is the difference between a function and a method in python?

- Function:- A function is an independent block of code used to perform a specific task. It can be called directly by its name. A method is also a function, but it is associated with an object or class and is called using the object name.

Example: print() is a function, while upper() is a method used with strings.

2. Explain the concept of function arguments and parameters in python.

- Parameters are the variables written in the function definition. Arguments are the actual values passed to the function when it is called.

Example: In add(a, b), a and b are parameters. In add(2,3), 2 and 3 are arguments.

3. What are the different ways to define and call a function in python?

- Functions in Python can be defined:
    - Without arguments
    - With arguments
    - With return values
    - Using lambda functions

Functions are called by writing the function name followed by parentheses.

Example: hello() or add(2,3)

4. What is the purpose of the return statement in a python function?

- The return statement is used to send the result of a function back to the caller. It ends the execution of the function and returns a value.

Example:

A function calculating the sum of two numbers can return the result using return.

5. What are iterators in python and how do they differ from iterables?

- An iterable is an object that can be looped over, such as lists, tuples, and strings. An iterator is an object that helps to access elements one by one during iteration.

Example:

A list is an iterable, while iter(list) creates an iterator.

6. Explain the concept of generators in python and how they are defined?

- Generators are special functions that generate values one at a time instead of returning all values at once. They are defined using the yield keyword.

Example:

A generator function can produce numbers one by one using yield.

7. What are the advantages of using generators over regular Functions?

- Advantages of Generators over Regular Functions
    - Generators use less memory.
    - They generate values only when needed.
    - They are faster for large datasets.
    - They make iteration easier.

Example:

A generator producing numbers from 1 to 1000 saves memory compared to storing all values at once.

8. What is a lambda function in Python and when is it typically used?

- A lambda function is a small anonymous function written in a single line. It is mainly used for short operations.

Example:

A lambda function can be used to calculate the square of a number.

9. Explain the purpose and usage of the map() function in Python.

- The map() function applies a given function to each element of an iterable and returns the modified result.

Example:

Using map() to multiply all numbers in a list by 2.

10. What is the difference between 'map(), reduce()', and 'filter() functions in Python?

- map() modifies all elements of a sequence.
- filter() selects elements based on a condition.
- reduce() combines all elements into a single value.

Example:

map() doubles numbers, filter() selects even numbers, and reduce() calculates the total sum.

11. Using pen & Paper write the internal mechanism for sum operation using reduce function on this given list: [47,11,42,13];

- The reduce() function performs operations step by step on list elements.

Process:

```
- First: 47 + 11 = 58
- Then: 58 + 42 = 100
- Finally: 100 + 13 = 113
```

So, the final result is 113.

```
# 1. Function to return sum of all even numbers in a list

def sum_even(numbers):
    return sum(num for num in numbers if num % 2 == 0)

nums = [1, 2, 3, 4, 5, 6]
print("Sum of even numbers:", sum_even(nums))
```

```
Sum of even numbers: 12
```

```
# 2. Function to reverse a string

def reverse_string(text):
    return text[::-1]

print(reverse_string("Python"))
```

```
nohtyP
```

```
# 3. Function to return squares of numbers in a list

def square_list(numbers):
    return [num ** 2 for num in numbers]

nums = [1, 2, 3, 4, 5]
print(square_list(nums))
```

```
[1, 4, 9, 16, 25]
```

```
# 4. Function to check prime numbers from 1 to 200

def is_prime(num):
    if num <= 1:
        return False
```

```
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False

    return True

for number in range(1, 201):
    if is_prime(number):
        print(number, end=" ")
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 1
```

```
# 5. Iterator class for Fibonacci sequence

class Fibonacci:

    def __init__(self, n):
        self.n = n
        self.a = 0
        self.b = 1
        self.count = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.count >= self.n:
            raise StopIteration

        value = self.a
        self.a, self.b = self.b, self.a + self.b
        self.count += 1

        return value

fib = Fibonacci(10)

for num in fib:
    print(num)
```

```
0
1
1
2
3
5
8
13
21
34
```

```
# 6. Generator function for powers of 2

def powers_of_two(n):
    for i in range(n + 1):
        yield 2 ** i

for value in powers_of_two(5):
    print(value)
```

```
1
2
4
8
16
32
```

```
# 7. Generator function to read file line by line

def read_file(file_name):
    with open(file_name, 'r') as file:
        for line in file:
            yield line.strip()

# Example
# for line in read_file("sample.txt"):
#     print(line)
```

```
# 8. Lambda function to sort tuples based on second element

data = [('a', 3), ('b', 1), ('c', 2)]

sorted_data = sorted(data, key=lambda x: x[1])

print(sorted_data)
```

```
[('b', 1), ('c', 2), ('a', 3)]
```

```
# 9. Convert Celsius to Fahrenheit using map()

celsius = [0, 20, 37, 100]

fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))

print(fahrenheit)
```

```
[32.0, 68.0, 98.6, 212.0]
```

```
# 10. Remove vowels from a string using filter()

text = "Hello Python"
```

```
result = ''.join(filter(lambda x: x.lower() not in 'aeiou', text))

print(result)
```

```
Hll Pythn
```

```
# 11. Book shop program using lambda and map()

orders = [
    [34587, "Learning Python, Mark Lutz", 4, 40.95],
    [98762, "Programming Python, Mark Lutz", 5, 56.80],
    [77226, "Head First Python, Paul Barry", 3, 32.95],
    [88112, "Einführung in Python3, Bernd Klein", 3, 24.99]
]

result = list(map(
    lambda item: (
        item[0],
        item[2] * item[3] + 10 if item[2] * item[3] < 100 else item[2] * item[3
    ),
    orders
))

print(result)
```

```
[(34587, 163.8), (98762, 284.0), (77226, 108.85000000000001), (88112, 84.97)]
```